Hochschule Bonn-Rhein-Sieg

arc42 Architekturbeschreibung:

WirSchiffenDas
Engine Analysis

Stand: 22. September 2026
Modul: Service-basierte Komponenten-Architekturen (SEKA)
Semester: Sommersemester 2026

Inhaltsverzeichnis

Abbildungsverzeichnis I

Tabellenverzeichnis II

1. Projektüberblick und Architekturziele ..... - 1 -

1.1 Ausgangssituation und Ziel ..... - 1 -

1.3 Fachliche Anforderungen ..... - 2 -

2. Randbedingungen ..... - 3 -

2.1 Projektgrenzen ..... - 3 -

3. Fachliche Kontextabgrenzung ..... - 4 -

4. Lösungsstrategie ..... - 5 -

5. Bausteinsicht ..... - 5 -

5.1 Systemübersicht ..... - 5 -

6. Laufzeitsicht ..... - 6 -

6.1 Ablauf einer regulären Analyse ..... - 6 -

6.2 Fehlerverhalten während der Analyse ..... - 9 -

6.2.1 Fachlicher Fehler ..... - 9 -

6.2.2 Technischer Ausfall ..... - 9 -

7. Verteilungssicht ..... - 9 -

7.1 Containerstruktur und Bereitstellung ..... - 10 -

8. Architekturentscheidungen ..... - 11 -

9. Qualitätsanforderungen und Architecture Smells ..... - 12 -

10. Risiken und technische Schulden ..... - 13 -

11. KI-Werkzeuge und Nutzungsrahmen ..... - 13 -

Literaturverzeichnis ..... - 15 -

Abbildungsverzeichnis

Abbildung 1: Kontextdiagramm des Optional Engine Configuration Analysis System (eigene Darstellung). ..... - 4 -

Abbildung 2: Komponentendiagramm der servicebasierten Analysearchitektur (eigene Darstellung). ..... - 6 -

Abbildung 3 Sequenzdiagramm des Analyseablaufs mit parallelen Teilanalysen und UI-Polling (eigene Darstellung). ..... - 8 -

Abbildung 4 Verteilungsdiagramm der Docker-basierten Systemarchitektur (eigene Darstellung) ..... - 10 -

Tabellenverzeichnis

Tabelle 1 Eingesetzte Technologien und deren Verwendung im Proof of Concept ..... - 3 -

Tabelle 2 Zentrale Architekturentscheidungen und ihre Begründung..... - 12 -

Tabelle 3 Relevante Architecture Smells und umgesetzte Gegenmaßnahmen ..... - 13 -

1. Projektüberblick und Architekturziele

1.1 Ausgangssituation und Ziel

Das Semesterprojekt untersucht die Modernisierung der Analysefunktion innerhalb des Bereichs Manufacturing Products der Fallstudie WirSchiffenDas. In der Ausgangssituation ist die Analyse eng gekoppelt und kann durch lange Simulationsläufe die übrige Produktkonstruktion blockieren [1]. Für das Optional Equipment eines Motors sollen fachlich getrennte Prüfungen ausgeführt werden, deren Bearbeitungszustand und Resultate für den Ingenieur sichtbar bleiben. Der Proof-of-Concept überführt die zentrale Analysefunktion in eine servicebasierte Architektur. Eine Motorkonfiguration wird über eine Weboberfläche erfasst und gespeichert. Anschließend werden unabhängige Teilanalysen für Fluid, Mechanical und Electrical parallel ausgeführt. Nach Abschluss dieser drei Teilanalysen bewertet Engine Management die Resultate und erzeugt das Gesamtergebnis. Die Anforderungen an persistente Konfigurationen, REST-Schnittstellen, proaktive Statusmeldungen und die Behandlung nicht erreichbarer Services leiten sich aus den Projektaufgaben ab [2].

1.2 Architekturziele

- Die Analysebereiche Fluid, Mechanical, Electrical und Engine Management werden als klar getrennte fachliche Einheiten umgesetzt. Jeder Service übernimmt dabei einen eigenen abgegrenzten Teil der Gesamtanalyse.
- Technische Störungen einzelner Analyse-Services sollen gezielt abgefangen werden. Durch den Einsatz von Circuit Breakern und Fallbacks werden Ausfälle erkannt und in definierte Fehlerzustände überführt, anstatt die gesamte Verarbeitung unkontrolliert scheitern zu lassen.
- Für jede gestartete Analyse soll der Bearbeitungsstand eindeutig nachvollziehbar sein. Über die jeweilige analysisId werden die Statusinformationen der einzelnen Services, deren Teilresultate sowie das abschließende Gesamtergebnis zusammengeführt und abrufbar gemacht.
- Die Kommunikation zwischen den Services erfolgt über festgelegte HTTP-/REST-Schnittstellen. Dadurch bleiben die internen Implementierungsdetails der einzelnen Services voneinander getrennt und direkte Abhängigkeiten werden reduziert.

- Die voneinander unabhängigen Teilanalysen sollen gleichzeitig ausgeführt werden können. Dadurch muss die Benutzeroberfläche nicht bis zum Abschluss aller Analysevorgänge blockieren und kann währenddessen weiterhin den aktuellen Status darstellen.
- Jeder Spring-Boot-Service wird als eigene Container-Einheit bereitgestellt. Docker Compose dient dazu, die fünf Container gemeinsam zu starten und deren lokale Zusammenarbeit zu ermöglichen.

1.3 Fachliche Anforderungen

- Für die einzelnen fachlichen Analysebereiche sollen mindestens vier voneinander abgegrenzte Microservices eingesetzt werden, die jeweils einen eigenen Prüf- bzw. Analysealgorithmus übernehmen.
- Konfigurationsdaten müssen erfasst, geprüft, dauerhaft gespeichert und zu einem späteren Zeitpunkt erneut geladen werden können.
- Der Fortschritt der einzelnen Analysevorgänge soll über definierte Zustände wie PENDING, RUNNING, READY und FAILED nachvollziehbar sein.
- Neben den Ergebnissen der einzelnen Analysebereiche soll auch ein zusammenfassendes Gesamtergebnis für die vollständige Analyse bereitgestellt werden.
- Treten technische Probleme bei einzelnen Analyse-Services auf, soll die Anwendung diese Ausfälle gezielt behandeln, ohne dass die gesamte Anwendung unkontrolliert abbricht.
- Der gesamte Proof-of-Concept soll containerisiert betrieben werden. Docker und Docker Compose dienen dabei zur Bereitstellung und gemeinsamen Ausführung der einzelnen Services.

2. Randbedingungen

Technologie	Einsatz im Projekt	Bezug
Spring Boot	REST-Services und Ablaufsteuerung	alle Services
HTML / CSS / JavaScript	Web UI und Statusdarstellung	configuration
H2 / Spring Data JPA	Konfigurationen und Analysezustände	configuration
Resilience4j	Circuit Breaker und Fallbacks	configuration
Docker / Docker Compose	Containerisierung und lokaler Betrieb	alle Services

Tabelle 1 Eingesetzte Technologien und deren Verwendung im Proof of Concept

2.1 Projektgrenzen

Der Proof of Concept setzt den Schwerpunkt auf die Architektur und das Zusammenspiel der beteiligten Services. Die fachlichen Analysefunktionen dienen deshalb in erster Linie dazu, den Ablauf der verteilten Verarbeitung demonstrierbar zu machen. Anstelle einer realitätsnahen Simulation des Motors kommen vereinfachte und reproduzierbare Prüfregeln zum Einsatz [2]. Nicht Gegenstand der Umsetzung sind Funktionen, die vor allem für einen späteren produktiven Einsatz notwendig wären. Dazu zählen insbesondere eine dauerhaft belastbare Datenhaltung, Zugriffsschutz durch Authentisierung und Autorisierung sowie eine zentrale Infrastruktur für Logging, Tracing und Monitoring.

3. Fachliche Kontextabgrenzung

Im fachlichen Umfeld des Systems steht der Ingenieur beziehungsweise Produktentwickler als zentraler Anwender im Mittelpunkt. Zu seinen Aufgaben gehören das Anlegen und Speichern einer Motorkonfiguration sowie das Auslösen des Analysevorgangs. Anschließend werden ihm sowohl der aktuelle Bearbeitungsstand als auch die Resultate der einzelnen Prüfungen und die abschließende Gesamtbewertung angezeigt. Die Geschäftsleitung beeinflusst das System durch übergeordnete Vorgaben und Zielsetzungen, während Manufacturing Products die produktbezogenen Anforderungen und fachlichen Rahmenbedingungen definiert. Technische Details der internen Service-Struktur bleiben in der Kontextsicht unberücksichtigt [3].

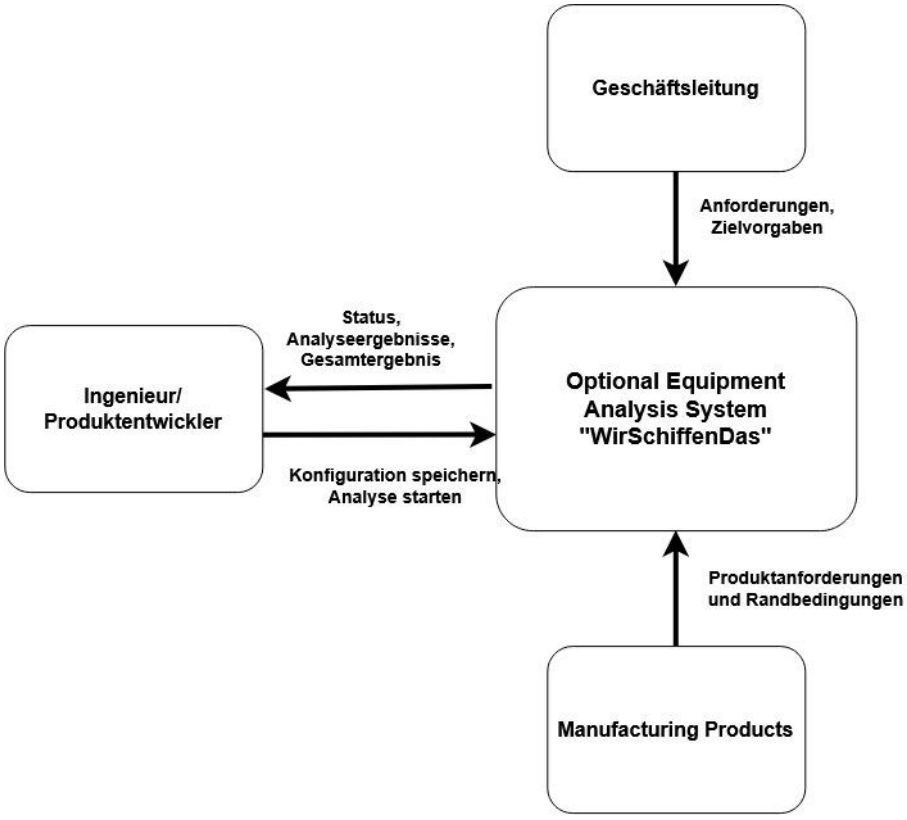


Abbildung 1: Kontextdiagramm des Optional Engine Configuration Analysis System (eigene Darstellung).

4. Lösungsstrategie

Der Proof of Concept ist fachlich nach den Analysebereichen Fluid, Mechanical, Electrical und Engine Management gegliedert. Dadurch bleiben zusammengehörige Aufgaben innerhalb eines Services gebündelt und Abhängigkeiten zwischen den Analysebereichen werden reduziert. Diese Art der Service-Zerlegung orientiert sich an fachlichen Fähigkeiten und unterstützt klar abgegrenzte Verantwortlichkeiten. Zugleich entspricht sie den Vorgaben der Projektaufgabe zur fachlichen Modellierung und zur Berücksichtigung geeigneter Servicegrenzen. Der configuration-Service übernimmt die zentrale Koordination. Er verwaltet Konfigurationen und Analysezustände, stellt die Weboberfläche bereit und startet die Analyse-Services. Fluid, Mechanical und Electrical werden parallel ausgeführt; anschließend folgt Engine Management auf Basis der vorherigen Ergebnisse. Status und Resultate werden über REST/HTTP zurückgemeldet und von der Weboberfläche regelmäßig abgefragt. REST ermöglicht dabei klar definierte und gut testbare Schnittstellen, führt jedoch zu einer stärkeren zeitlichen Kopplung als eine asynchrone Nachrichtenkommunikation.

5. Bausteinsicht

5.1 Systemübersicht

m Zentrum der Architektur steht der configuration-Service. Er stellt die Weboberfläche bereit, verwaltet Konfigurationen und Analysen in der H2-Datenbank und koordiniert die Kommunikation mit den vier Analyse-Services. Die Analyseaufträge werden über REST an Fluid, Mechanical, Electrical und Engine Management gesendet. Die Analyse-Services besitzen keine eigene Persistenz, sondern melden ihren aktuellen Status sowie das jeweilige Ergebnis über einen Callback an den configuration-Service zurück. Dort werden die Rückmeldungen gespeichert und für die weitere Ablaufsteuerung sowie die Anzeige in der Weboberfläche verwendet.

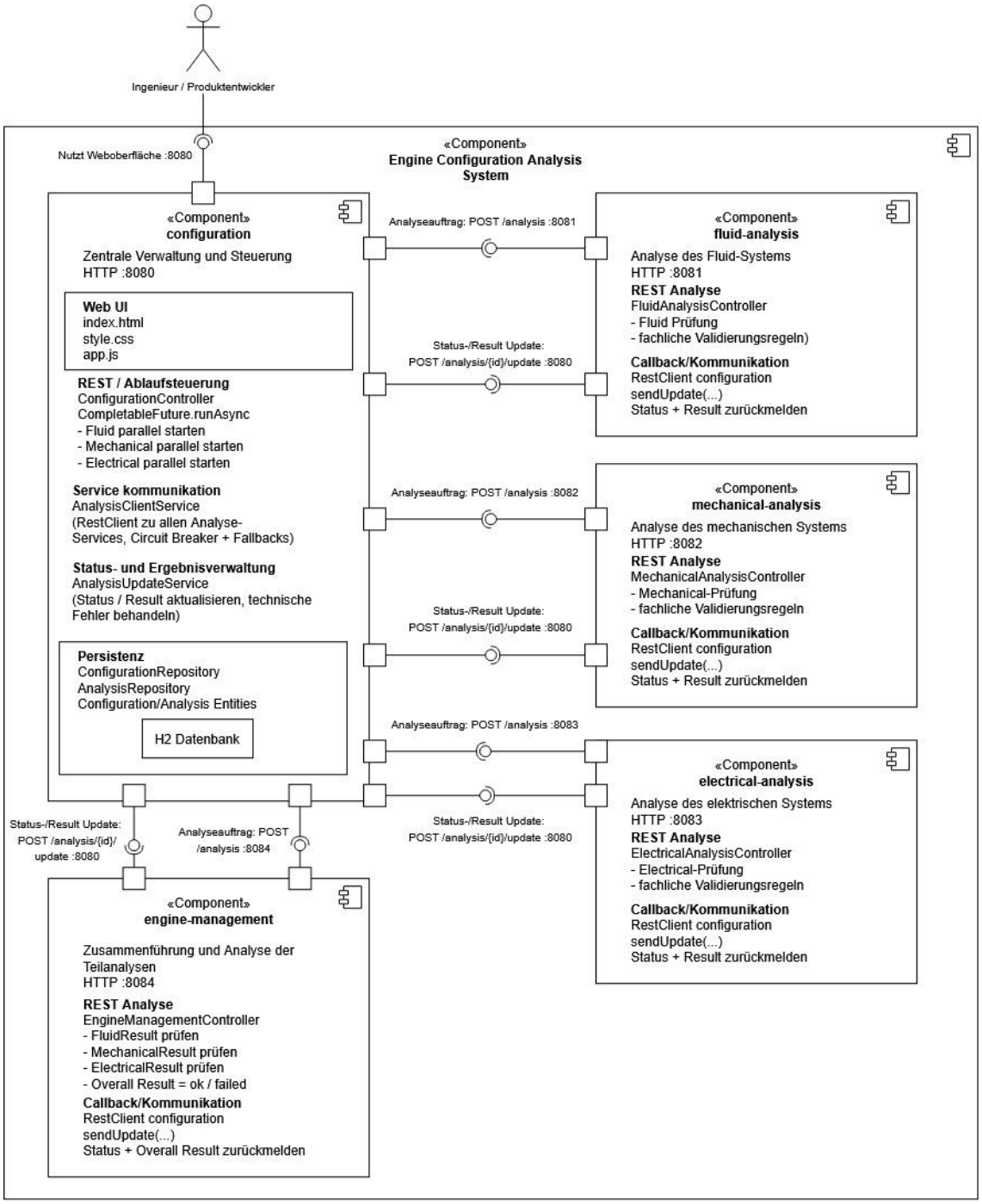


Abbildung 2: Komponentendiagramm der servicebasierten Analysearchitektur (eigene Darstellung).

6. Laufzeitsicht

6.1 Ablauf einer regulären Analyse

Nach dem Start einer Analyse legt der configuration-Service einen neuen Analysevorgang an und speichert dessen Zustand. Anschließend werden die drei unabhängigen Services fluid-analysis, mechanical-analysis und electrical-analysis parallel ausgeführt. Jeder dieser

Services meldet seinen Bearbeitungsstatus und das jeweilige fachliche Ergebnis an den configuration-Service zurück. Sobald alle drei Teilanalysen abgeschlossen sind, wird der engine-management-Service gestartet. Dieser verarbeitet die bereits vorliegenden Ergebnisse und erzeugt daraus das abschließende Gesamtergebnis. Während des gesamten Ablaufs fragt die Weboberfläche regelmäßig den aktuellen Stand der Analyse ab und stellt Status sowie Ergebnisse fortlaufend dar.

1. Über die Weboberfläche wird die Analyse für eine zuvor gespeicherte Motorkonfiguration gestartet.
2. Der configuration-Service erstellt den Analysevorgang und stößt Fluid, Mechanical und Electrical parallel an.
3. Die drei Analyse-Services melden zunächst den Zustand running und nach Abschluss ready zusammen mit ihrem jeweiligen Resultat zurück.
4. Nach Beendigung aller drei Teilanalysen startet der configuration-Service die Engine-Management-Analyse und übergibt die bereits vorhandenen Ergebnisse.
5. engine-management wertet diese Informationen aus und übermittelt sein Ergebnis an den configuration-Service.
6. Das Gesamtergebnis wird gespeichert und zusammen mit den Teilresultaten in der Weboberfläche angezeigt.

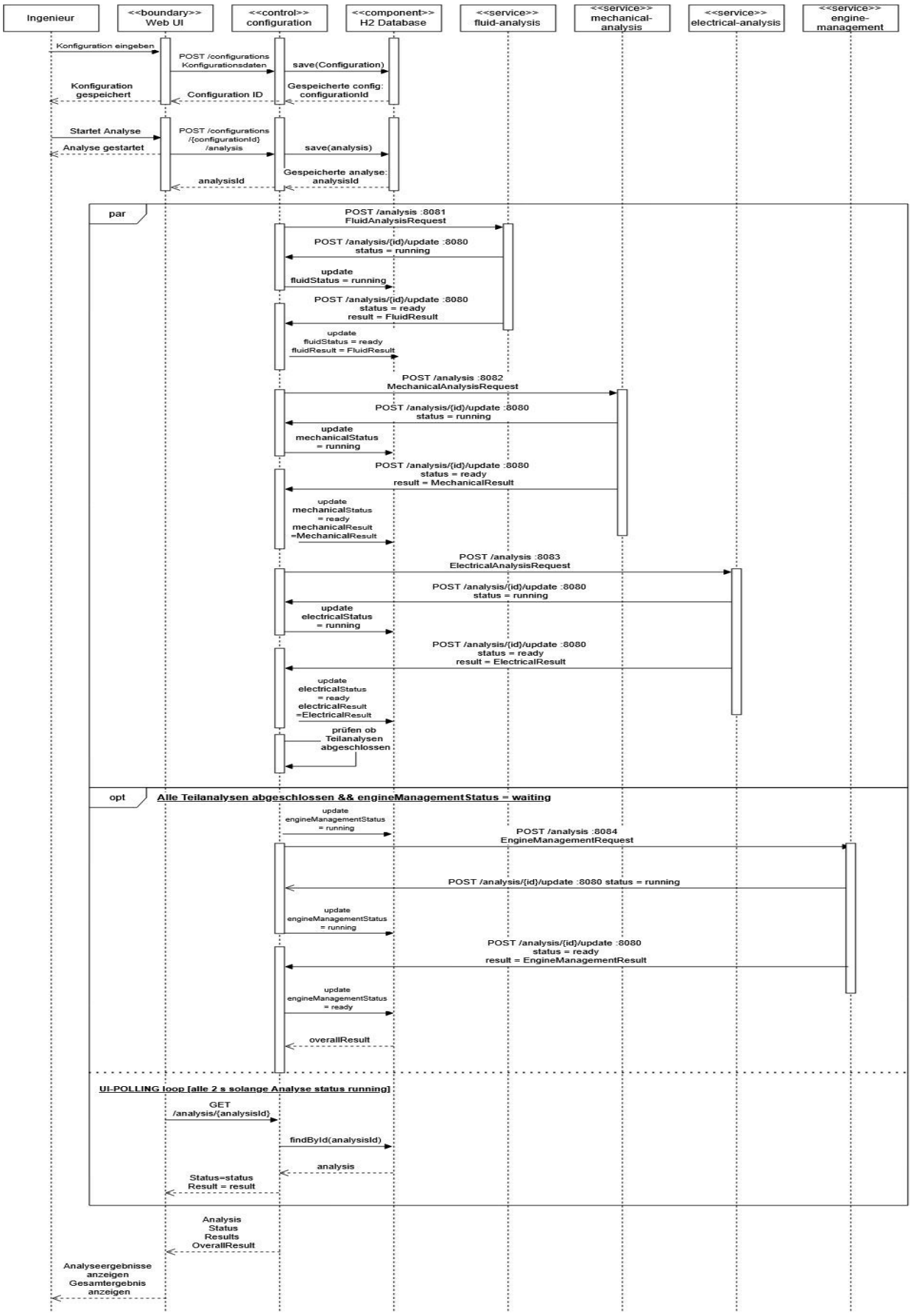


Abbildung 3 Sequenzdiagramm des Analyseablaufs mit parallelen Teilanalysen und UI-Polling (eigene Darstellung).

6.2 Fehlerverhalten während der Analyse

6.2.1 Fachlicher Fehler

Ein Analyse-Service ist erreichbar und führt seine Prüfung vollständig aus, erkennt dabei jedoch eine ungültige Kombination. Ein Beispiel ist die Mechanical Analysis: Wird powerTransmission = hybrid gemeinsam mit gearboxOptions = manual gewählt, bewertet der Service die Getriebeprüfung als failed. Technisch wurde die Analyse trotzdem erfolgreich beendet, weshalb der Status des Services auf ready gesetzt wird. Das fachliche Resultat enthält dagegen den Fehler. Der Circuit Breaker wird in diesem Fall nicht ausgelöst, da die Kommunikation mit dem Service funktioniert hat. Das negative Teilresultat fließt anschließend in die Engine-Management-Analyse ein und kann zu einem negativen Gesamtergebnis führen.

6.2.2 Technischer Ausfall

Ein anderer Ablauf entsteht, wenn ein Analyse-Service beispielsweise nicht erreichbar ist. Ist etwa der Container von mechanical-analysis gestoppt, schlägt der REST-Aufruf des configuration-Services an diesen Service technisch fehl. Dieser Aufruf wird durch Resilience4j überwacht. Der Fehler wird vom Circuit Breaker registriert und die zugehörige Fallback-Methode ausgeführt. Dadurch wird die Mechanical Analysis kontrolliert auf den Status failed gesetzt, ein reguläres fachliches Ergebnis liegt in diesem Fall nicht vor. Der Circuit Breaker reagiert also nicht auf ein fachliches failed, sondern auf technische Fehler wie einen nicht erreichbaren Service oder einen fehlgeschlagenen REST-Aufruf. Sobald die konfigurierte Fehlerschwelle erreicht ist, öffnet sich der Circuit Breaker und weitere Aufrufe an den betroffenen Service werden vorübergehend direkt über den Fallback behandelt.

7. Verteilungssicht

Die Anwendung wird lokal über Docker Compose ausgeführt. Jeder der fünf Spring-Boot-Services läuft in einem eigenen Container. Der configuration-Service übernimmt dabei zusätzlich die Bereitstellung der Weboberfläche und enthält die eingebettete H2-Datenbank. Eine separate Datenbank- oder Frontend-Instanz wird daher nicht betrieben.

Innerhalb der Docker-Umgebung erreichen sich die Services über ihre Compose-Service-Namen. Dadurch müssen für die interne Kommunikation keine festen IP-Adressen verwendet werden. Der Zugriff des Benutzers erfolgt über den Browser auf Port 8080 des configuration-Containers.

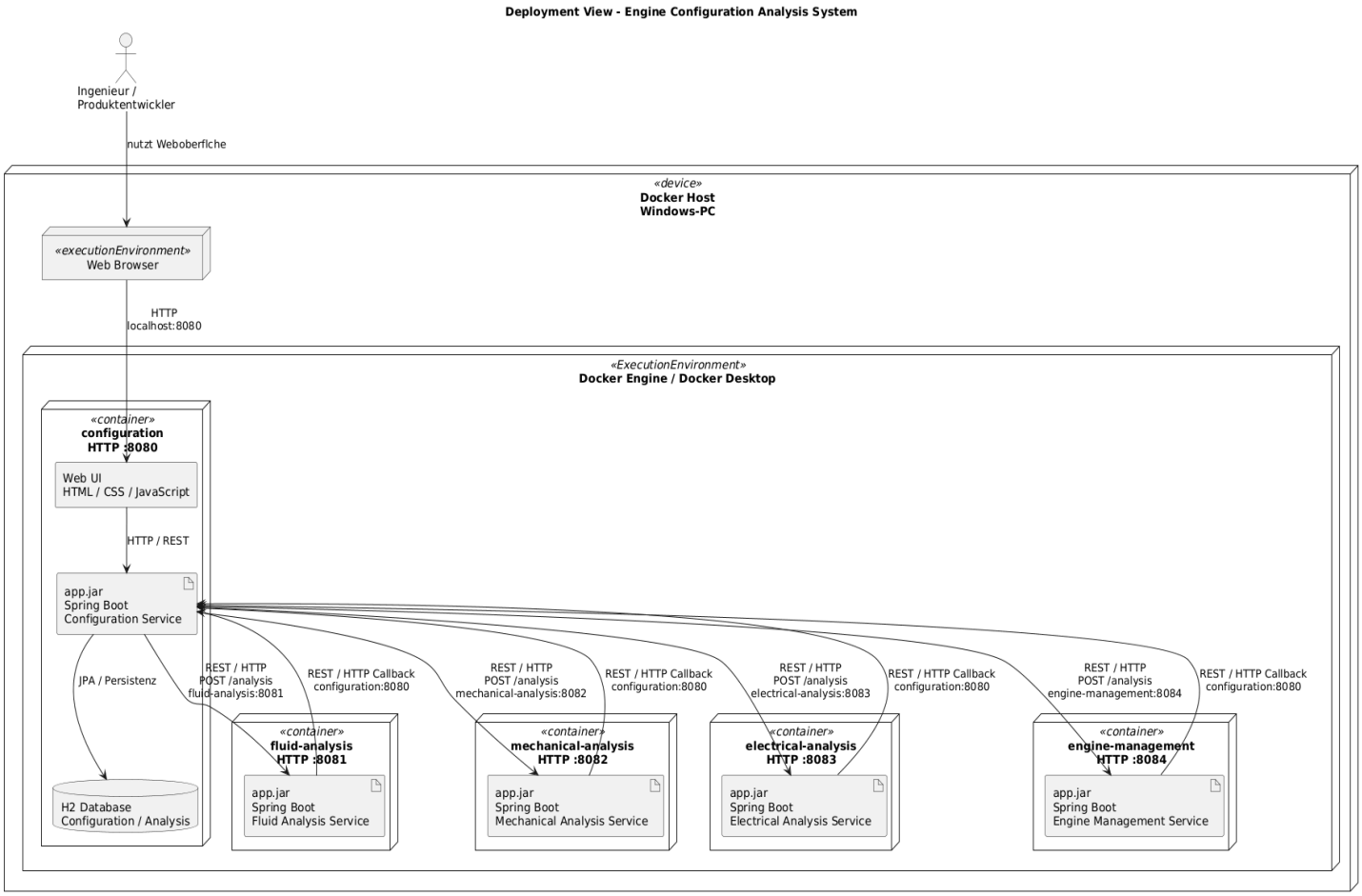


Abbildung 4 Verteilungsdiagramm der Docker-basierten Systemarchitektur (eigene Darstellung)

- Jeder Microservice wird als eigenständiger Docker-Container ausgeführt.
- Die Analyse-Services besitzen keine eigene Datenhaltung und können daher unabhängig gestartet oder gestoppt werden.
- Weboberfläche, Koordinationslogik und H2-Datenbank befinden sich gemeinsam im configuration-Container.
- Die interne Service-Kommunikation erfolgt über Docker-Compose-Service-Namen wie fluid-analysis, mechanical-analysis oder engine-management.
- Docker Compose startet und verbindet alle fünf Container als gemeinsame lokale Laufzeitumgebung.

- Technische Service-Adressen werden über Umgebungsvariablen gesetzt, sodass die Anwendung innerhalb der Container nicht von localhost-Adressen abhängig ist.

8. Architekturentscheidungen

Architekturentscheidung	Begründung
Zentrale Ablaufsteuerung im configuration-Service	Configuration startet Analysen, sammelt Callbacks und entscheidet über den Start von Engine Management. Für den kleinen PoC ist der Ablauf dadurch leicht nachvollziehbar und demonstrierbar.
Parallelisierung mit CompletableFuture	Die voneinander unabhängigen Teilanalysen werden parallel gestartet, um die UI und den Gesamtprozess nicht unnötig durch eine sequenzielle Ausführung zu blockieren.
REST/HTTP als Integrationsstil	Alle Servicegrenzen werden über explizite REST-Endpunkte angesprochen. Das vereinfacht Test und Demonstration, akzeptiert aber synchrone zeitliche Kopplung.
Aufteilung der Analyse-Services	Die Analyse wird nach Fluid, Mechanical, Electrical und Engine Management getrennt. Dies stärkt Kohäsion und macht die fachlichen Verantwortlichkeiten im Modell sichtbar.
Zustandslose Analyse-Services mit zentraler Speicherung	Die Analyse-Services erhalten Konfigurationsdaten und müssen selbst keine dauerhaften Zustände verwalten. Dadurch bleiben sie einfach gehalten und können unabhängig voneinander gestartet, beendet oder ersetzt werden.

Circuit Breaker für technische Service-Ausfälle	Fehler bei REST-Aufrufen werden kontrolliert behandelt. Fallbacks sorgen dafür, dass ein nicht erreichbarer Analyse-Service in einen definierten Fehlerzustand überführt wird.
Getrennte Docker-Container für die fünf Services	Die Services bleiben als eigene Laufzeiteinheiten voneinander getrennt und können gemeinsam über Docker Compose bereitgestellt werden.

Tabelle 2 Zentrale Architekturentscheidungen und ihre Begründung

9. Qualitätsanforderungen und Architecture Smells

Architekturproblem	Bedeutung im Projekt	Umsetzung / Gegenmaßnahme
Fehlende Fehlerisolation	Der Ausfall eines Analyse-Services könnte den gesamten Analyseablauf beeinträchtigen.	REST-Aufrufe werden mit Resilience4j Circuit Breakern abgesichert. Fallbacks überführen technische Fehler in einen definierten Zustand.
Statisch gebundene Service-Adressen	Fest eingetragene Netzwerkadressen würden die Services stark an eine bestimmte Laufzeitumgebung koppeln.	Innerhalb von Docker Compose werden Service-Namen und Umgebungsvariablen für die Kommunikation verwendet.
Unzureichende Beobachtbarkeit	Eine rein technische Aufteilung nach Schichten würde fachlich zusammengehörige Analysefunktionen auseinanderziehen.	Fluid, Mechanical, Electrical und Engine Management bilden jeweils einen eigenen fachlichen Analysebereich.

Gemeinsam genutzte Persistenz zwischen Analyse-Services	Eine gemeinsam verwendete Datenbank würde zusätzliche Abhängigkeiten zwischen den Analyse-Services erzeugen.	Die vier Analyse-Services bleiben zustandslos. Persistiert wird ausschließlich über den configuration-Service.

Tabelle 3 Relevante Architecture Smells und umgesetzte Gegenmaßnahmen

10. Risiken und technische Schulden

Der aktuelle Proof of Concept erfüllt die wesentlichen Anforderungen an den Analyseablauf, weist jedoch noch einige Einschränkungen auf. Eine zentrale Abhängigkeit besteht zum configuration-Service, da dort Koordination, Persistenz und die Bereitstellung der Weboberfläche zusammenlaufen. Ein Ausfall dieses Services hätte daher unmittelbare Auswirkungen auf große Teile des Systems. Hinzu kommt, dass die verwendete H2-Datenbank vor allem für die Demonstration geeignet ist und keine produktionsreife Datenhaltung darstellt. Auch die Kommunikation über REST führt zu einer gewissen zeitlichen Kopplung, da die beteiligten Services zum Zeitpunkt des Aufrufs erreichbar sein müssen [1]. Technische Ausfälle werden zwar durch Circuit Breaker und Fallbacks kontrolliert behandelt, dennoch bleibt die Architektur an mehreren Stellen von der Erreichbarkeit einzelner Komponenten abhängig. Darüber hinaus beschränkt sich die Beobachtbarkeit aktuell auf Statusinformationen und Ergebnisse in der Benutzeroberfläche. Zentralisiertes Logging, Distributed Tracing und technische Metriken fehlen ebenso wie Authentisierung und Autorisierung. Diese Punkte stellen daher sinnvolle Erweiterungen dar, falls der Proof of Concept später in Richtung eines produktiven Systems weiterentwickelt werden soll.

11. KI-Werkzeuge und Nutzungsrahmen

Im Rahmen des Semesterprojekts wurde künstliche Intelligenz als unterstützendes Hilfsmittel eingesetzt. Die im Projekt verwendeten KI-Werkzeuge hatten ausschließlich einer unterstützenden Funktion. Bewertung der Ergebnisse, die Auswahl der Architektur sowie die endgültige Ausgestaltung von Texten, Modellen und sonstigen Projektergebnissen lagen beim Verfasser. Es wurde ausschließlich ChatGPT genutzt, um

Code zu erläutern und zu überarbeiten sowie bei der sprachlichen Überarbeitung und Kontrolle der Architekturdokumentation. Sämtliche durch KI erzeugten Inhalte wurden vor ihrer Verwendung inhaltlich überprüft und überarbeitet. Personenbezogene oder vertrauliche Dokumente und Daten wurden dabei nicht an die eingesetzten Systeme weitergegeben. Die KI diente ausschließlich als unterstützendes Hilfsmittel und ersetzte weder die eigenständige Bearbeitung des Themas noch die fachliche Kontrolle der Ergebnisse. Die vorliegende Arbeit wurde selbstständig erstellt.

Literaturverzeichnis

[1] S. Alda, „Case Study Microservices - Analyse der Architektur der Firma WirSchiffenDas,“ Hochschule Bonn-Rhein-Sieg, Fachbereich Informatik, Sankt Augustin, 2026.

[2] S. Alda, „Übungsblätter 4–6 zur Vorlesung Service-basierte Komponenten-Architekturen (SEKA),“ Hochschule Bonn-Rhein-Sieg, Fachbereich Informatik, Sankt Augustin, 2026.

[3] G. Starke, Effektive Softwarearchitekturen: Ein praktischer Leitfaden, München: Carl Hanser Verlag, 2020.

[4] S. Newman, Building Microservices: Designing Fine-Grained Systems, Sebastopol: O’Reilly Media, 2015.